Ex. No: 11	ADD iOS AS A BUILD TARGET
Date of Exercise	
Git-hub Host Link	

Aim:

To configure a Flutter application for the iOS platform and understand the process of building and testing the application using Xcode and an iOS Simulator.

Description:

Flutter supports cross-platform application development, including iOS. The iOS build target allows developers to configure, build, and test Flutter applications on iOS devices. The iOS platform files are generated using the `flutter create` command. The application can be opened in Xcode, tested using an iOS Simulator, and built for iOS distribution. Actual iOS building and testing require macOS and Xcode.

Procedure:

Step 1: Check Flutter Installation

Execute the following command to check the Flutter installation and development environment.

```
flutter doctor
```

Step 2: Check Available Devices

List the connected devices using:

```
flutter devices
```

Step 3: Create a Flutter Project

Create a new Flutter application named `ios_app`.

```
flutter create ios_app
cd ios_app
```

Step 4: Verify iOS Platform Files

Check the generated iOS project files.

```
dir ios
```

The project contains the `Runner`, `Runner.xcodeproj`, and `Runner.xcworkspace` directories.

Step 5: Analyze the Application

Check the application code for errors using:

```
flutter analyze
```

The analysis completed successfully with the message `No issues found!.`

Step 6: Open the iOS Project in Xcode

On a macOS system, open the iOS workspace using:

```
open ios/Runner.xcworkspace
```

Step 7: Select an iOS Simulator

Open Xcode, select an available iOS Simulator, and configure the application for execution.

Step 8: Run the Application

Execute the application using:

```
flutter run
```

Alternatively, run the application directly from Xcode.

Step 9: Test the Application

Verify the following application features:

- User interface
- Navigation
- Text fields
- Buttons
- Images
- Screen transitions

Step 10: Build the iOS Application

Build the application for iOS using Flutter on a supported macOS environment.

```
flutter build ios
```

Program:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

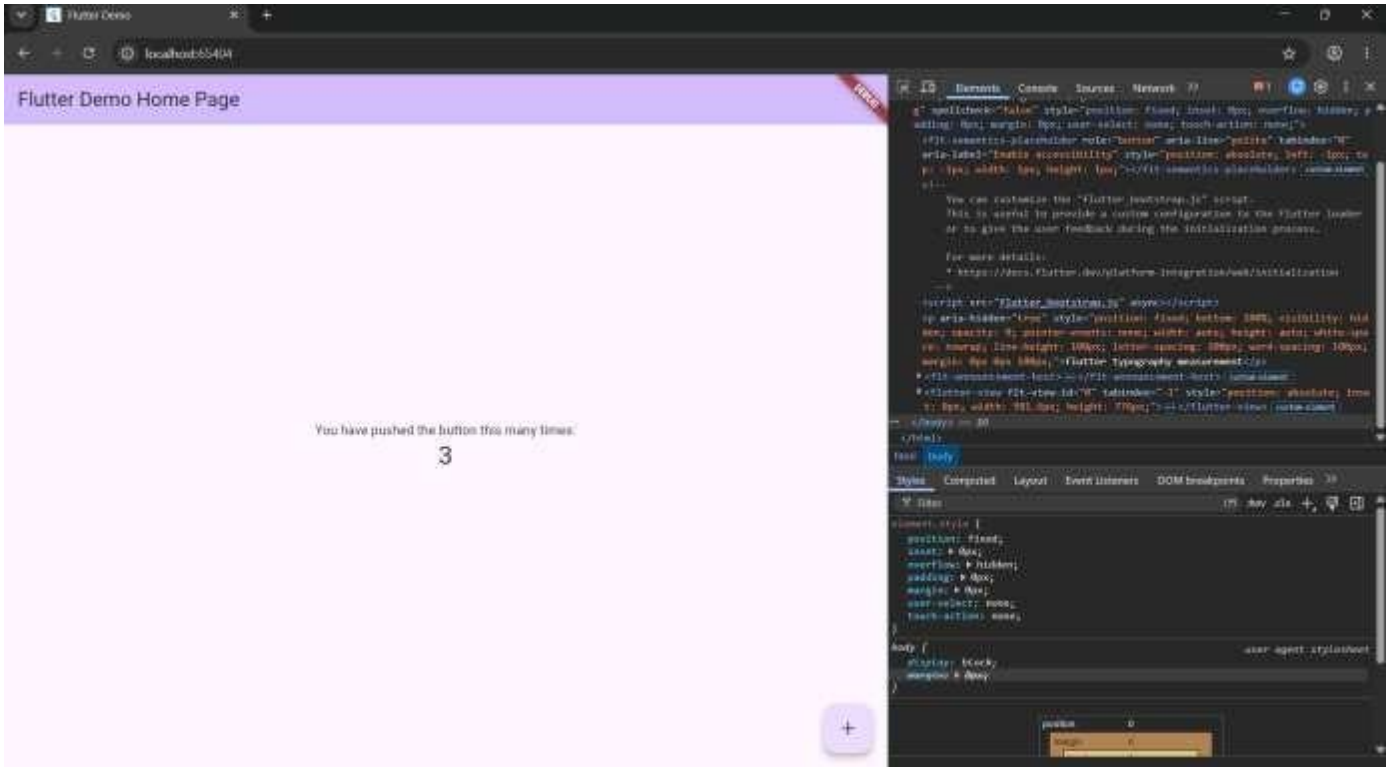
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
    debugShowCheckedModeBanner: false,
    title: 'iOS App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: const HomePage(),
    );
  }
}

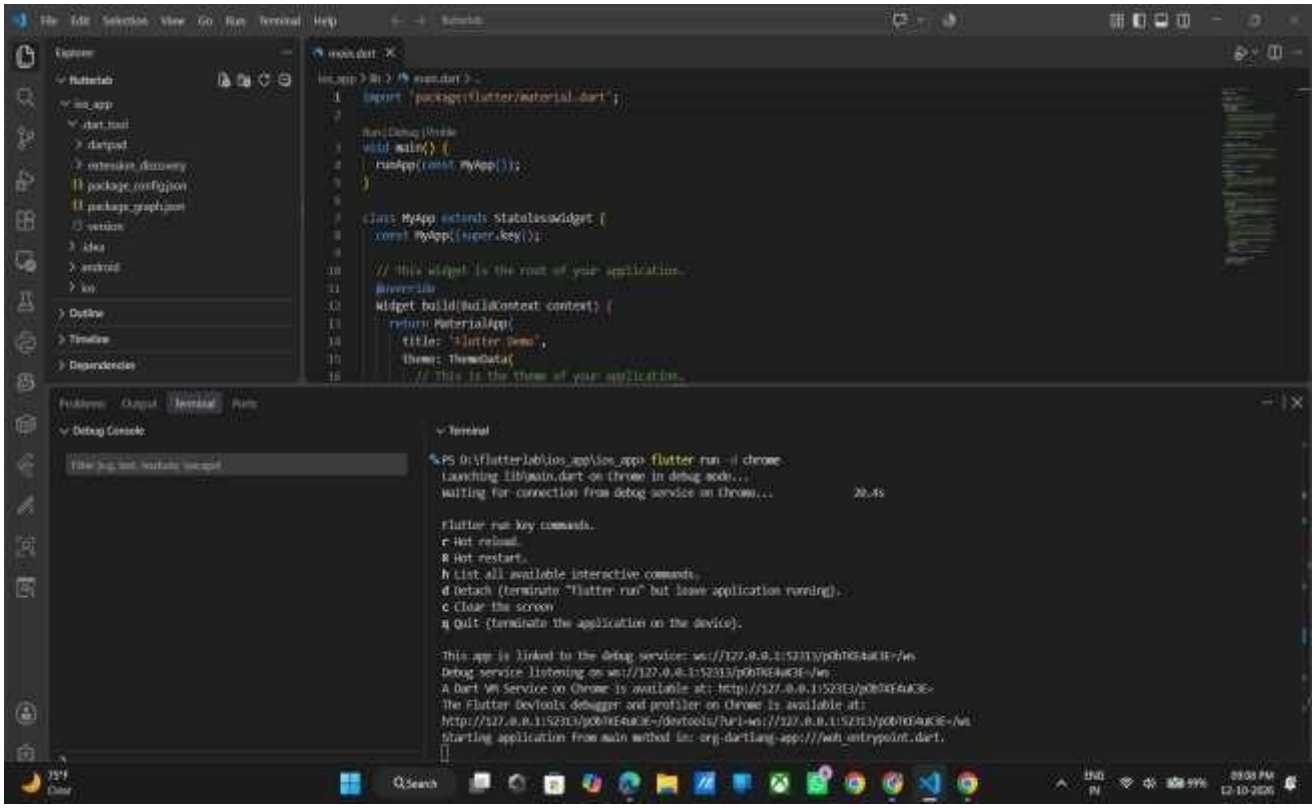
class HomePage extends StatelessWidget {
  const HomePage({super.key});

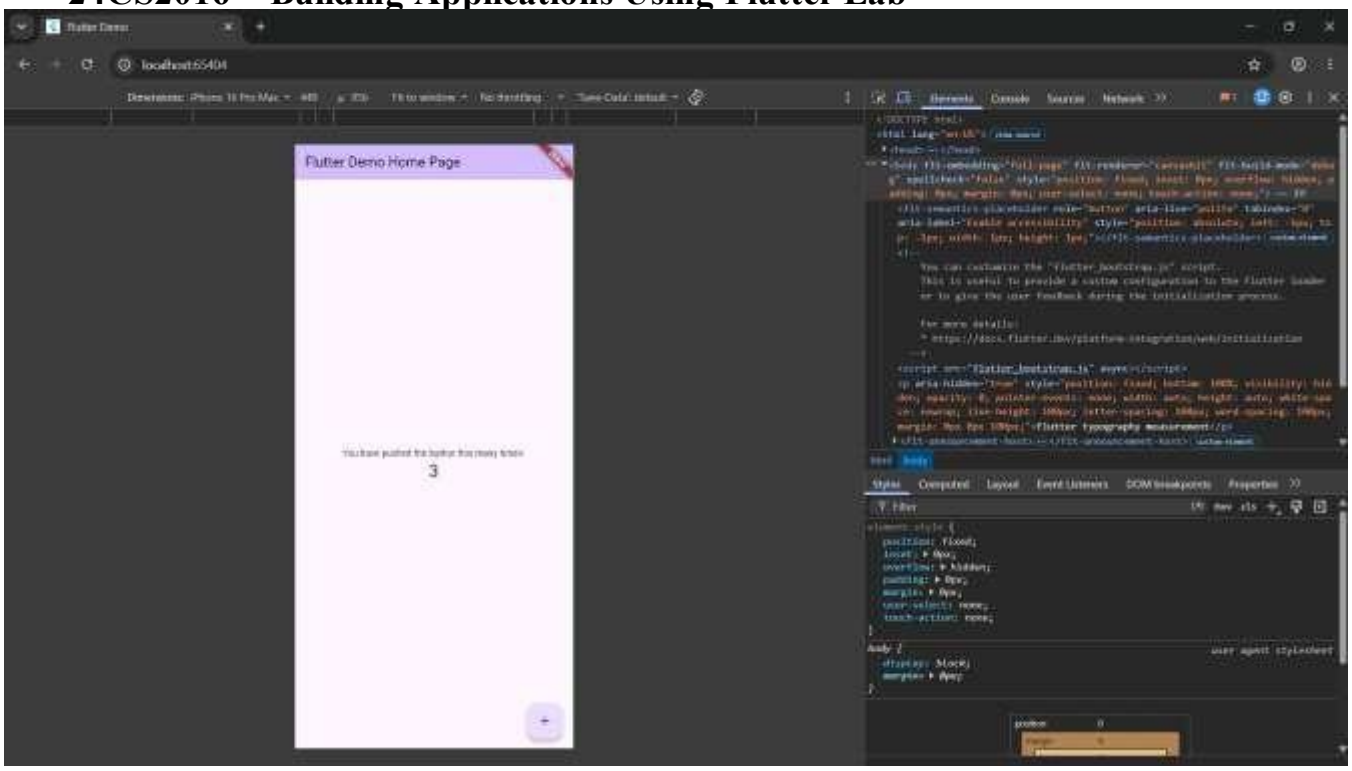
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('iOS Build Target'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(
              Icons.phone_iphone,
              size: 80,
              color: Colors.blue,
            ),
            const SizedBox(height: 20),
            const Text(
              'Welcome to Flutter iOS App',
              style: TextStyle(
```

```
            fontSize: 22,
            fontWeight: FontWeight.bold,
          ),
        ),
        const SizedBox(height: 20),
        ElevatedButton(
          onPressed: () {
            ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(
                content: Text('Application is working!'),
              ),
            );
          },
          child: const Text('Test Application'),
        ),
      ],
    ),
  ),
 );
 }
}
```

Output:




main.dart
import 'package:flutter/material.dart';
void main() {
runApp(const MyApp());
}
class MyApp extends StatelessWidget {
const MyApp({super.key});
// This widget is the root of your application.
@override
Widget build(BuildContext context) {
return MaterialApp(
title: 'Flutter Demo',
Terminal
PS D:\flutterlab\ios_app\ios_app> flutter run -d chrome
Launching lib\main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome...
Flutter run key commands.
r Hot reload.
R Hot restart.
h List all available interactive commands.
d Detach (terminate "flutter run" but leave application running).
c Clear the screen
q Quit (terminate the application on the device).
Starting application from main method in: org-dartlang-app:///web_entrypoint.dart.



RESULT

The Flutter project was created successfully, and the iOS platform configuration files were generated. Static code analysis was completed successfully without any issues. Actual iOS building and simulator testing require macOS and Xcode.